Scheduling Multithreaded Computations by Work Stealing

Robert D. Blumofe
The University of Texas at Austin, Austin, Texas

Charles E. Leiserson
MIT Laboratory for Computer Science, Cambridge, Massachusetts

Journal of the ACM, Vol. 46, No. 5, September 1999, pp. 720–748.

Abstract

This paper studies the problem of efficiently scheduling fully strict (i.e., well-structured) multithreaded computations on parallel computers. A popular and practical method of scheduling this kind of dynamic MIMD-style computation is "work stealing," in which processors needing work steal computational threads from other processors. In this paper, we give the first provably good work-stealing scheduler for multithreaded computations with dependencies.

Specifically, our analysis shows that the expected time to execute a fully strict computation on P processors using our work-stealing scheduler is $T_1/P + O(T_\infty)$, where T_1 is the minimum serial execution time of the multithreaded computation and T_∞ is the minimum execution time with an infinite number of processors. Moreover, the space required by the execution is at most $S_1 P$, where S_1 is the minimum serial space requirement. We also show that the expected total communication of the algorithm is at most $O(PT_\infty(1 + n_d)S_{\max})$, where $S_{\max}$ is the size of the largest activation record of any thread and n_d is the maximum number of times that any thread synchronizes with its parent.

This communication bound justifies the folk wisdom that work-stealing schedulers are more communication efficient than their work-sharing counterparts. All three of these bounds are existentially optimal to within a constant factor.

Categories and Subject Descriptors: D.4.1 [Process Management]: Threads—Thread scheduling
General Terms: Algorithms, Performance, Theory

This research was supported in part by the Advanced Research Projects Agency under Contract N00014-94-1-0985. This research was done while Robert D. Blumofe was at the MIT Laboratory for Computer Science and was supported in part by an ARPA High-Performance Computing Graduate Fellowship.

An earlier version of this paper appeared in the *Proceedings of the 35th Annual Symposium on Foundations of Computer Science* (November 20–22). IEEE Computer Society Press, Los Alamitos, Calif., 1994, pp. 356–368.

Authors' addresses: R. D. Blumofe, Department of Computer Sciences, Taylor Hall 2.124, The University of Texas at Austin, Austin, TX 78712; C. E. Leiserson, 545 Technology Square, Cambridge, MA 02139.

1 Introduction

For efficient execution of a dynamically growing "multithreaded" computation on a MIMD-style parallel computer, a scheduling algorithm must ensure that enough threads are active concurrently to keep the processors busy. Simultaneously, it should ensure that the number of concurrently active threads remains within reasonable limits so that memory requirements are not unduly large. Moreover, the scheduler should also try to maintain related threads on the same processor, if possible, so that communication between them can be minimized. Needless to say, achieving all these goals simultaneously can be difficult.

Two scheduling paradigms have arisen to address the problem of scheduling multithreaded computations: work sharing and work stealing. In work sharing, whenever a processor generates new threads,

the scheduler attempts to migrate some of them to other processors in hopes of distributing the work to underutilized processors. In work stealing, however, underutilized processors take the initiative: they attempt to "steal" threads from other processors. Intuitively, the migration of threads occurs less frequently with work stealing than with work sharing, since when all processors have work to do, no threads are migrated by a work-stealing scheduler, but threads are always migrated by a work-sharing scheduler.

The work-stealing idea dates back at least as far as Burton and Sleep's [1981] research on parallel execution of functional programs and Halstead's [1984] implementation of Multilisp. These authors point out the heuristic benefits of work stealing with regards to space and communication. Since then, many researchers have implemented variants on this strategy.[1] Rudolph et al. [1991] analyzed a randomized work-stealing strategy for load balancing independent jobs on a parallel computer, and Karp and Zhang [1993] analyzed a randomized work-stealing strategy for parallel backtrack search. Zhang and Ortynski [1994] have obtained good bounds on the communication requirements of this algorithm.

In this paper, we present and analyze a work-stealing algorithm for scheduling "fully strict" (well-structured) multithreaded computations. This class of computations encompasses both backtrack search computations [Karp and Zhang 1993; Zhang and Ortynski 1994] and divide-and-conquer computations [Wu and Kung 1991], as well as dataflow computations [Arvind et al. 1989] in which threads may stall due to a data dependency. We analyze our algorithms in a stringent atomic-access model similar to the atomic message-passing model of Liu et al. [1993] in which concurrent accesses to the same data structure are serially queued by an adversary.

Our main contribution is a randomized work-stealing scheduling algorithm for fully strict multithreaded computations which is provably efficient in terms of time, space, and communication. We prove that the expected time to execute a fully strict computation on P processors using our work-stealing scheduler is $T_1/P + O(T_\infty)$, where T_1 is the minimum serial execution time of the multithreaded computation and T_∞ is the minimum execution time with an infinite number of processors. In addition, the space required by the execution is at most $S_1 P$, where S_1 is the minimum serial space requirement. These bounds are better than previous bounds for work-sharing schedulers [Blumofe and Leiserson 1998], and the work-stealing scheduler is much simpler and eminently practical. Part of this improvement is due to our focusing on fully strict computations, as compared to the (general) strict computations studied in Blumofe and Leiserson [1998]. We also prove that the expected total communication of the execution is at most $O(PT_\infty(1 + n_d)S_{\max})$, where $S_{\max}$ is the size of the largest activation record of any thread and n_d is the maximum number of times that any thread synchronizes with its parent. This bound is existentially tight to within a constant factor, meeting the lower bound of Wu and Kung [1991] for communication in parallel divide-and-conquer. In contrast, work-sharing schedulers have nearly worst-case behavior for communication. Thus, our results bolster the folk wisdom that work stealing is superior to work sharing.

Others have studied and continue to study the problem of efficiently managing the space requirements of parallel computations. Culler and Arvind [1988] and Ruggiero and Sargeant [1987] give heuristics for limiting the space required by dataflow programs. Burton [1988] shows how to limit space in certain parallel computations without causing deadlock. More recently, Burton [1996] has developed and analyzed a scheduling algorithm with provably good time and space bounds. Blelloch et al. [1995; 1997] have also recently developed and analyzed scheduling algorithms with provably good time and space bounds. It is not yet clear whether any of these algorithms are as practical as work stealing.

The remainder of this paper is organized as follows: In Section 2, we review the graph-theoretic model of multithreaded computations introduced in Blumofe and Leiserson [1998], which provides a theoretical basis for analyzing schedulers. Section 3 gives a simple scheduling algorithm which uses a central queue. This "busy-leaves" algorithm forms the basis for our randomized work-stealing algorithm, which we present in Section 4. In Section 5, we introduce the atomic-access model that we use to analyze execution time and communication costs for the work-stealing algorithm, and we present and analyze a combinatorial "balls and bins" game that we use to derive a bound on the contention that arises in random work stealing. We then use this bound along with a delay-sequence argument [Ranade 1987] in Section 6 to analyze the execution time and communication cost of the work-stealing algorithm. To conclude, in Section 7, we briefly discuss how the theoretical ideas in this paper have been applied to the Cilk programming language and runtime system [Blumofe et al. 1996c; Frigo et al. 1998], as well as make some concluding remarks.

[1] See, for example, Blumofe and Lisiecki [1997], Feldmann et al. [1993], Finkel and Manber [1987], Halbherr et al. [1994], Kuszmaul [1994], Mohr et al. [1991], and Vandevoorde and Roberts [1988].

2 A Model of Multithreaded Computation

This section reprises the graph-theoretic model of multithreaded computation introduced in Blumofe and Leiserson [1998]. We also define what it means for computations to be "fully strict." We conclude with a statement of the greedy-scheduling theorem, which is an adaptation of theorems by Brent [1974] and Graham [1966; 1969] on dag scheduling.

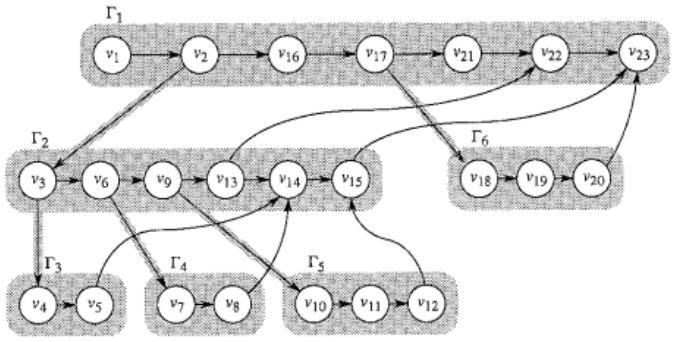

FIG. 1. A multithreaded computation. This computation contains 23 instructions v_1, v_2, ..., v_{23} and 6 threads Γ_1, Γ_2, ..., Γ_6.

Figure 1: A multithreaded computation. This computation contains 23 instructions $v_1, v_2, \ldots, v_{23}$ and 6 threads $\Gamma_1, \Gamma_2, \ldots, \Gamma_6$.

A multithreaded computation is composed of a set of threads, each of which is a sequential ordering of unit-time instructions. The instructions are connected by dependency edges, which provide a partial ordering on which instructions must execute before which other instructions. In Figure 1, for example, each shaded block is a thread with circles representing instructions and the horizontal edges, called continue edges, representing the sequential ordering. Thread T_5 of this example contains 3 instructions: v_{10}, v_{11}, and v_{12}. The instructions of a thread must execute in this sequential order from the first (leftmost) instruction to the last (rightmost) instruction. In order to execute a thread, we allocate for it a chunk of memory, called an activation frame, that the instructions of the thread can use to store the values on which they compute.

A P-processor execution schedule for a multithreaded computation determines which processors of a P-processor parallel computer execute which instructions at each step. An execution schedule depends on the particular multithreaded computation and the number P of processors. In any given step of an execution schedule, each processor executes at most one instruction.

During the course of its execution, a thread may create, or spawn, other threads. Spawning a thread is like a subroutine call, except that the spawning thread can operate concurrently with the spawned thread. We consider spawned threads to be children of the thread that did the spawning, and a thread may spawn as many children as it desires. In this way, threads are organized into a spawn tree as indicated in Figure 1 by the downward-pointing, shaded dependency edges, called spawn edges, that connect threads to their spawned children.

The spawn tree is the parallel analog of a call tree. In our example computation, the spawn tree's root thread Γ_1 has two children, Γ_2 and Γ_6, and thread Γ_2 has three children, Γ_3, Γ_4, and Γ_5. Threads Γ_3, Γ_4, Γ_5, and Γ_6, which have no children, are leaf threads.

Each spawn edge goes from a specific instruction—the instruction that actually does the spawn operation—in the parent thread to the first instruction of the child thread. An execution schedule must obey this edge in that no processor may execute an instruction in a spawned child thread until after the spawning instruction in the parent thread has been executed. In our example computation (Figure 1), due to the spawn edge (v_6, v_7), instruction v_7 cannot be executed until after the spawning instruction v_6. Consistent with our unit-time model of instructions, a single instruction may spawn at most one child. When the spawning instruction executes, it allocates an activation frame for the new child thread. Once a thread has been spawned and its frame has been allocated, we say the thread is alive or living. When the last instruction of a thread executes, it deallocates its frame and the thread dies.

An execution schedule generally respects other dependencies besides those represented by continue and spawn edges. Consider an instruction that produces a data value to be consumed by another

instruction. Such a producer/consumer relationship precludes the consuming instruction from executing until after the producing instruction. To enforce such orderings, other dependency edges, called join edges, may be required, as shown in Figure 1 by the curved edges. If the execution of a thread arrives at a consuming instruction before the producing instruction has executed, execution of the consuming thread cannot continue—the thread stalls. Once the producing instruction executes, the join dependency is resolved, which enables the consuming thread to resume its execution—the thread becomes ready. A multithreaded computation does not model the means by which join dependencies get resolved or by which unresolved join dependencies get detected. In implementation, resolution and detection can be accomplished using mechanisms such as join counters [Blumofe et al. 1996c], futures [Halstead 1984], or I-structures [Arvind et al. 1989].

We make two technical assumptions regarding join edges. We first assume that each instruction has at most a constant number of join edges incident on it. This assumption is consistent with our unit-time model of instructions. The second assumption is that no join edges enter the instruction immediately following a spawn. This assumption means that when a parent thread spawns a child thread, the parent cannot immediately stall. It continues to be ready to execute for at least one more instruction.

An execution schedule must obey the constraints given by the spawn, continue, and join edges of the computation. These dependency edges form a directed graph of instructions, and no processor may execute an instruction until after all of the instruction's predecessors in this graph have been executed. So that execution schedules exist, this graph must be acyclic. That is, it must be a directed acyclic graph, or dag. At any given step of an execution schedule, an instruction is ready if all of its predecessors in the dag have been executed.

We make the simplifying assumption that a parent thread remains alive until all its children die, and thus, a thread does not deallocate its activation frame until all its children's frames have been deallocated. Although this assumption is not absolutely necessary, it gives the execution a natural structure, and it will simplify our analyses of space utilization. In accounting for space utilization, we also assume that the frames hold all the values used by the computation; there is no global storage available to the computation outside the frames (or if such storage is available, then we do not account for it). Therefore, the space used at a given time in executing a computation is the total size of all frames used by all living threads at that time, and the total space used in executing a computation is the maximum such value over the course of the execution.

To summarize, a multithreaded computation can be viewed as a dag of instructions connected by dependency edges. The instructions are connected by continue edges into threads, and the threads form a spawn tree with the spawn edges. When a thread is spawned, an activation frame is allocated and this frame remains allocated as long as the thread remains alive. A living thread may be either ready or stalled due to an unresolved dependency.